

Trabajo: Técnicas de Reconocimiento de Patrones

Jorge Bengoa Pinedo

Bogurad Barański Barańska

Sergio Izquierdo Morona

8 de febrero de 2026

Índice

1. Introducción a la detección de objetos en imágenes térmicas	2
1.1. Estado del Arte en detección térmica	2
1.2. Redes Neuronales Convolucionales (CNN)	2
1.3. Metodología YOLO	3
2. Fundamentos de funcionamiento de la red YOLO	3
2.1. Neurona de la red YOLO	3
2.2. Topología de la red	5
2.2.1. Backbone: Extracción de Características	10
2.2.2. Neck: Fusión de Escalas	10
2.2.3. Head: Detección	10
2.3. Reglas de entrenamiento	12
2.3.1. Entrenamiento de la red	12
2.3.2. Hiperparámetros	13
3. Desarrollo: Aplicación en Imágenes Térmicas	16
4. Resultados	16

1. Introducción a la detección de objetos en imágenes térmicas

Hoy en día con el avance de la automatización en muchas aplicaciones es de vital importancia la percepción del entorno mediante visión artificial como pueden ser aplicaciones de seguridad o sistemas de conducción autónoma. Aunque el estándar en muchos casos sean imágenes con cámaras convencionales (RGB), en escenarios con mala iluminación como momentos con niebla o por la noche su rendimiento decae drásticamente. Por ello, para trabajar con estos contextos se utiliza la termografía infrarroja (TIR) como alternativa robusta capaz de capturar la radiación emitida por los cuerpos independientemente de la luz ambiental [1].

No obstante, el procesamiento de imágenes térmicas presenta desafíos únicos que no aparecen en imágenes convencionales:

- Baja resolución motivada por el alto coste de los sensores. Esta limitación no es un problema en el conjunto de datos empleados en este trabajo.
- Falta de información de color al solo tener un canal de información, texturas difusas y bajo contraste entre el objeto y el fondo, lo que genera efecto halo [1].

Para abordar este problema de detección y clasificación, en este trabajo se estudia la arquitectura YOLOv5 (You Only Look Once) [2], validando su desempeño sobre el conjunto de datos estándar para sistemas avanzados de asistencia al conductor proporcionado por Teledyne FLIR [3].

1.1. Estado del Arte en detección térmica

Históricamente, la detección de objetos en imágenes infrarrojas se basaba en técnicas de visión computacional tradicionales que utilizaban características hechas a mano. Métodos como el Histograma de Gradientes Orientados (HOG) combinados con Máquinas de Soporte Vectorial (SVM) eran comunes [4] [5], pero carecían de capacidad de generalización y robustez frente a la variabilidad térmica [1].

La revolución del aprendizaje profundo provocó que estas técnicas quedarán desfasadas propiciando el uso de Redes Neuronales Convolucionales (CNNs). En la literatura actual, los detectores modernos se dividen en dos categorías principales:

- **Detectores de dos etapas:** Modelos como R-CNN y sus variantes (Fast R-CNN, Faster R-CNN) [6]. Estos algoritmos generan primero una serie de propuestas de región y luego las clasifican. Aunque alcanzan una gran precisión, su alta carga computacional dificulta su implementación en tiempo real, un requisito indispensable en aplicaciones como la conducción autónoma.
- **Detectores de una etapa:** Modelos como SSD (Single Shot Multibox Detector) [7] y la familia YOLO [8]. Estos procesan la imagen y predicen las cajas delimitadoras en una única evaluación de la red, logrando un equilibrio óptimo entre velocidad y precisión.

Estudios recientes [1], han demostrado que las arquitecturas basadas en YOLO, específicamente optimizadas para la banda infrarroja (imágenes térmicas), superan a los métodos de dos etapas en escenarios dinámicos donde la latencia es crítica.

1.2. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) constituyen la base matemática de los sistemas de visión modernos. A diferencia de las redes neuronales densas clásicas (MLP), las CNNs están diseñadas para procesar datos con topología de rejilla (como imágenes), empleando la operación de convolución para extraer características espaciales de manera jerárquica.

El principio fundamental reside en el uso de filtros o kernels que va aprendiendo la red que se deslizan sobre la imagen de entrada, generando mapas de características que representan patrones visuales de complejidad creciente: desde bordes y texturas simples en las capas iniciales, hasta formas y objetos completos en las capas profundas. Esta capacidad de extracción automática de características elimina la necesidad de diseñar descriptores manuales, permitiendo al modelo aprender las representaciones más relevantes directamente de los datos térmicos.

1.3. Metodología YOLO

La metodología YOLO redefine el problema de la detección de objetos como un único problema de regresión, en contraposición al enfoque de clasificación de ventanas deslizantes. La arquitectura divide la imagen de entrada en una rejilla de $S \times S$, donde cada celda es responsable de predecir objetos si el centro de estos esta dentro.

Para el desarrollo del proyecto hemos seleccionado la arquitectura YOLOv5 [2]. Aunque este modelo integra mejoras estructurales muy potentes como el backbone CSP-Darknet y el cuello PANet (detallados en la sección de topología de la red), la razón principal de esta elección ha sido su implementación nativa en Python. Al no depender del framework clásico en C, se puede agilizar la integración con otras librerías y simplificar considerablemente el flujo de trabajo.

2. Fundamentos de funcionamiento de la red YOLO

Para comprender cómo funciona YOLO, analizaremos los tres pilares que integran su estructura:

- Caracterización de la neurona.
- Definición de una topología de interconexión.
- Definición de unas reglas de entrenamiento.

2.1. Neurona de la red YOLO

La unidad fundamental de procesamiento en la arquitectura YOLO es la capa convolucional (típica de las CNN). En esta sección se describe el funcionamiento de dicha operación, tomando como caso de estudio la capa de entrada, analizando tanto el entrenamiento específico realizado para imágenes térmicas como la implementación por defecto, tal como se ilustra en la Figura 1.

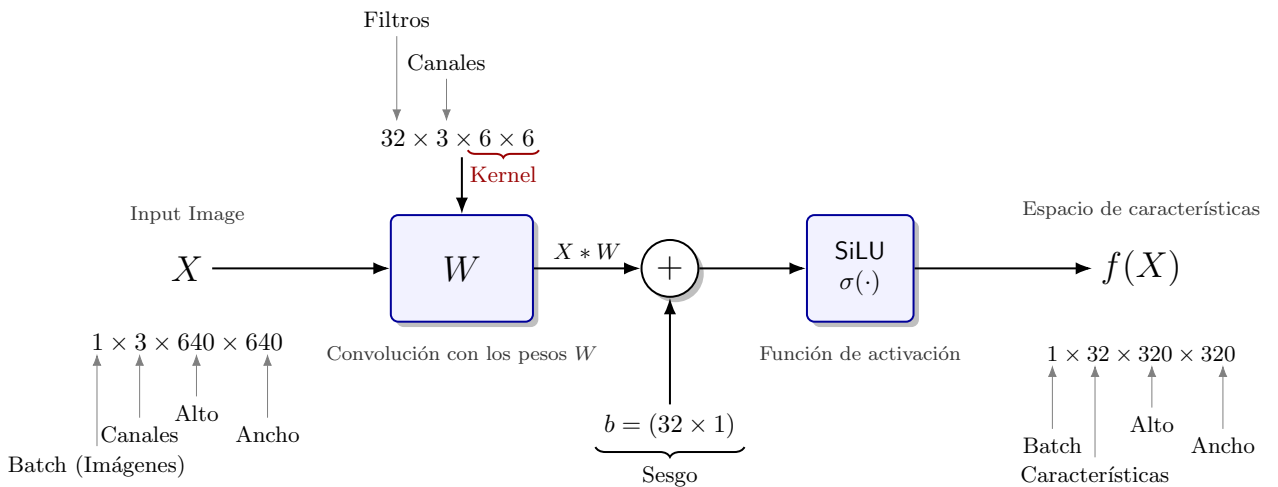


Figura 1: Estructura de la neurona a la entrada de la red YOLO.

La operación de esta neurona convolucional está definida por los siguientes hiperparámetros:

- **Número de filtros (Características C_{out}):** Determinado por la primera dimensión del tensor de pesos. Define cuántos mapas de características se generarán.
- **Canales de entrada (C_{in}):** Determinado por la segunda dimensión del tensor de pesos (e.g., 3 para RGB, 1 para imágenes térmicas).
- **Tamaño del Kernel (k):** Define las dimensiones espaciales de la ventana de convolución (últimas dos dimensiones de los pesos). En la arquitectura YOLO analizada, se emplean principalmente los tamaños (6,6) para la entrada, y (3,3) o (1,1) para capas profundas.
- **Padding (P):** Parámetro de relleno utilizado para controlar las dimensiones espaciales de la salida e indicar los bordes al llenarlos de ceros. En YOLO se utilizan típicamente valores de (2,2), (1,1), o (0,0).
- **Stride (S):** Controla el paso o desplazamiento del filtro sobre la imagen. Un $S = 1$ mueve el filtro píxel a píxel (preservando resolución), mientras que $S = 2$ lo desplaza cada dos píxeles, reduciendo la dimensión espacial a la mitad.
- **Dilatación (d):** Controla la separación entre los puntos que analiza el kernel (también conocida como *atrous convolution* cuando $d > 1$). En esta implementación de YOLO, el parámetro es (1,1), lo que indica una convolución estándar compacta sin huecos entre los elementos del kernel.
- **Agrupación (g):** Controla la conectividad entre los canales de entrada y salida. Dado que en YOLO se utiliza $g = 1$, se realiza una convolución estándar donde todos los canales de entrada contribuyen al cálculo de cada filtro de salida.

A partir de estos hiperparámetros, es posible determinar las dimensiones espaciales del mapa de características de salida. Dado que se contempla el factor de dilatación, se debe utilizar la relación generalizada descrita por [9].

$$\text{Salida} = \left\lfloor \frac{\text{Entrada} + 2 \cdot P - [k + (k - 1)(d - 1)]}{S} \right\rfloor + 1 \quad (1)$$

En cuanto a la activación de cada neurona, el proceso consiste en aplicar la función SiLU (*Sigmoid Linear Unit*) al resultado de la operación lineal (convolución más sesgo). Matemáticamente, la activación $f(X)$ se expresa como:

$$f(X) = \underbrace{[(W * X) + b]}_z \cdot \underbrace{\sigma([(W * X) + b])}_z \quad (2)$$

donde $\sigma(z) = \frac{1}{1+e^{-z}}$ es la función sigmoide logística. Se utiliza esta función de activación porque permite el cálculo de gradientes y solo permite valores negativos pequeños para mejorar el flujo de gradientes [10, 11], se ilustra en la Figura 2.

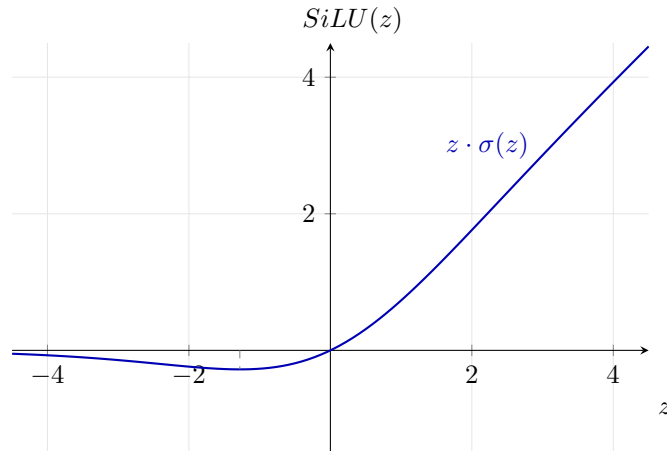


Figura 2: Representación gráfica de la función de activación SiLU.

Por otro lado, la operación de convolución discreta 2D ($W * X$) en el contexto de aprendizaje profundo se implementa técnicamente como una correlación cruzada (sin voltear el kernel) pues solo se busca conocer los

pesos y no aplicar un filtro con una topología concreta. Para un filtro de salida específico (c_{out}) en la posición espacial (h, w), la operación se define formalmente como [9]:

$$(W * X)(c_{out}, h, w) = \sum_{l=0}^{\frac{c_{in}}{g}-1} \sum_{i=0}^{K_H-1} \sum_{j=0}^{K_W-1} W(c_{out}, l, i, j) \cdot X(l, h \cdot S + i \cdot d, w \cdot S + j \cdot d) \quad (3)$$

2.2. Topología de la red

En esta sección se discute la topología de la red tras analizar los artículos principales de los creadores de YOLO [8][12][13][14] y su diagrama de conexión mediante la herramienta *Netron* donde se introduce la red en formato *.onnx* para su análisis, un ejemplo de parte de la red se puede ver en la figura 3.

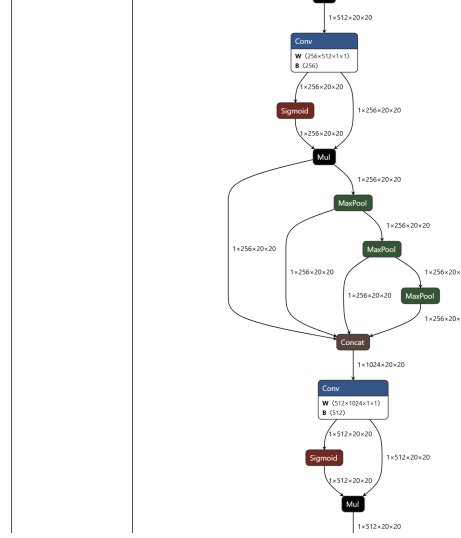


Figura 3: Ejemplo de parte de la red YOLO en la aplicación *Netron*.

Uno de los bloques que más se repite a lo largo de la arquitectura de la red YOLO es el bloque ResNet, cuya estructura se encuentra representada en la figura 4. La filosofía detrás del diseño de este bloque viene detallada en el artículo [15], y su objetivo principal se mejoró el rendimiento de las redes neuronales profundas cuando se acumulan múltiples capas.

La problemática surge durante el entrenamiento: al acumular múltiples capas consecutivas, los gradientes tienden a explotar. Antes de la existencia de este bloque, esto se mitigaba mediante la normalización en capas intermedias, pero aun así se observaba una degradación en la convergencia. Para entender esta degradación, se puede entender la operación de convolución como una aplicación identidad con cierto ruido superpuesto que representa las características a extraer. Por tanto, mediante una conexión lineal directa se optimiza el comportamiento del método de optimización utilizado, mejorando así la convergencia.

La premisa de que los bloques se comportan como aplicaciones lineales con ruido se demuestra empíricamente en el propio artículo [15]. Aunque se trata más de una heurística que de un argumento formal, en casos reales, como el de la figura 5, presenta un mejor comportamiento que simplemente acumular capas, y solo sería problemático si el objetivo del bloque fuera aproximar la función nula.

La interpretación del bloque ResNet como una aplicación lineal con ruido se deduce fácilmente a partir de las ecuaciones del modelo:

$$y = \mathcal{H}(x) = \mathcal{F}(x) + x \quad (4)$$

donde y es la salida de aplicar $\mathcal{H}$ a la entrada x que se descompone entre una función que representa la convolución $\mathcal{F}(x)$ y la aplicación lineal directa. Por tanto, se puede asemejar el funcionamiento de las capas convolucionales como un extractor del ruido o características.

Profundizando en el detalle del propio bloque, este está formado por dos capas, donde la primera se encarga de procesar los canales para que la segunda simplemente extraiga las características de los anteriores.

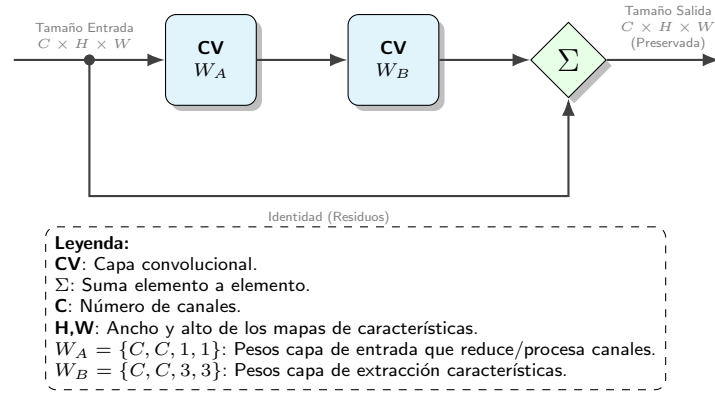


Figura 4: Arquitectura de un bloque ResNet.

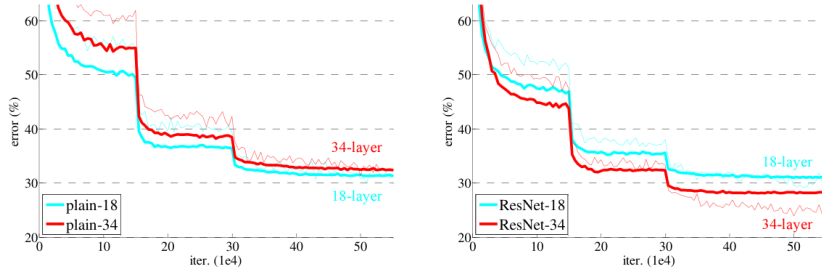


Figura 5: Representación del error de entrenamiento en función del número de capas. A la izquierda se representa una arquitectura con capas apiladas y a la derecha una arquitectura de capas apiladas utilizando el bloque ResNet. Figura extraída de [15].

El siguiente bloque básico dentro de la arquitectura YOLO es el bloque CSP o C3. La filosofía detrás del bloque representado en la figura 6 se basa en el artículo [16]. Este componente, se diseña para mejorar la ineficiencia dentro de las redes convolucionales asociada a la duplicidad de información en el gradiente. Esta problemática, se debe a que los gradientes de retropropagación se copian y repiten a través de las diferentes capas, lo que genera cálculos redundantes que no contribuyen significativamente al aprendizaje de características nuevas. Para mitigarlo la arquitectura CSP envía el mapa de características de entrada a través de los canales base C para procesarlo en dos ramas en paralelo.

Como se observa en la figura 6, tras la concatenación de entrada, el flujo se bifurca. Una rama principal procesa la información mediante una secuencia de capas convolucionales (representadas por los pesos W_1 , W_2 y W_3), mientras que una rama parcial conecta directamente la entrada con el final del bloque a través de una única capa (peso W_1 inferior). Esta división permite que el gradiente se propague por dos caminos distintos, aumentando la variabilidad del aprendizaje y reduciendo la cantidad de operaciones. Por tanto, dada una entrada x , la red procesa obtiene el tensor de salida y como la concatenación de las dos ramas en paralelo:

$$y = \mathcal{T}[\mathcal{F}(x) || \mathcal{G}(x)] \quad (5)$$

donde $\mathcal{T}$ es la convolución de la capa de salida que sirve para fusionar la información proveniente de los dos canales concatenados $\mathcal{F}(x) || \mathcal{G}(x)$, la transformación $\mathcal{F}(x)$ realizada por la capa convolucional W_1 sirve para fusionar la información proveniente de los dos canales de entrada y la transformación $\mathcal{G}(x)$ consiste en extraer características mediante la capa convolucional W_3 que trabaja con un mapa de características comprimido en la capa W_2 .

Por tanto, este bloque es una forma de poder reciclar el cálculo de gradientes de compresión de canales para poder tener una capa especializada únicamente en la extracción de características W_3 . Para finalmente en la capa W_4 combinar el contexto global de la imagen proveniente de la rama inferior con las características extraídas en la capa superior.

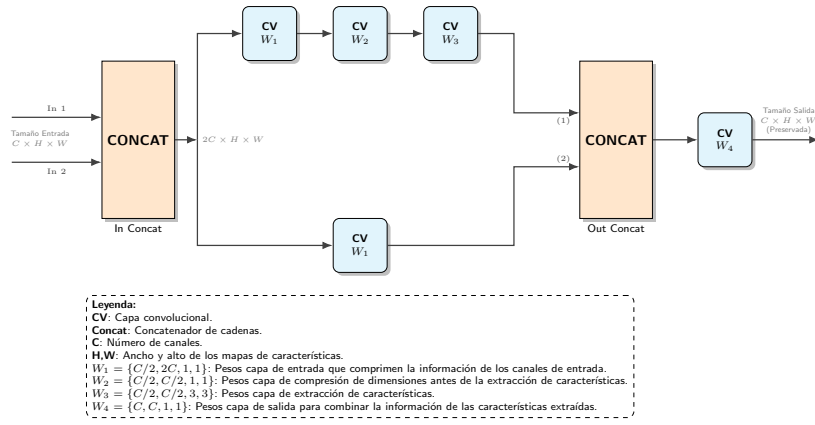


Figura 6: Arquitectura del bloque CSP o C3.

Una variante fundamental de la arquitectura CSP es el bloque CSPResNe(X)t o C3-k, el cual sustituye las secuencias convolucionales estándar por bloques ResNet para potenciar la extracción de características. Su representación esquemática se detalla en la figura 7, y su diseño se fundamenta nuevamente en el artículo [16].

La filosofía de este diseño reside en la concatenación en serie de bloques ResNet, repetidos un número k de veces según la complejidad de las características a modelar. Esto convierte al bloque C3-k en el mecanismo principal para controlar la profundidad de la red YOLO. Al concentrar la mayor densidad de procesamiento, este módulo se especializa en la extracción profunda de características.

En consecuencia, la integración de los módulos ResNet resulta crítica para la estabilidad del entrenamiento. Como se ha discutido previamente, las conexiones residuales mitigan el desvanecimiento del gradiente, permitiendo que el modelo converja y aprenda adecuadamente incluso al aumentar la profundidad de la red.

Analizando el esquema, es relevante destacar la función específica de cada etapa: la capa de entrada W_1 actúa como un primer extractor de características superficiales, generando dos proyecciones del canal de entrada (reduciendo la resolución del espacio de características). Posteriormente, las capas W_2 se encargan de comprimir estos canales: una rama profundiza en la extracción mediante la serie de k bloques ResNet, mientras que la otra preserva la señal original. Finalmente, tras la concatenación, la capa W_3 fusiona ambos flujos, integrando los detalles finos aprendidos por la red profunda con la estructura general preservada por la rama inferior.

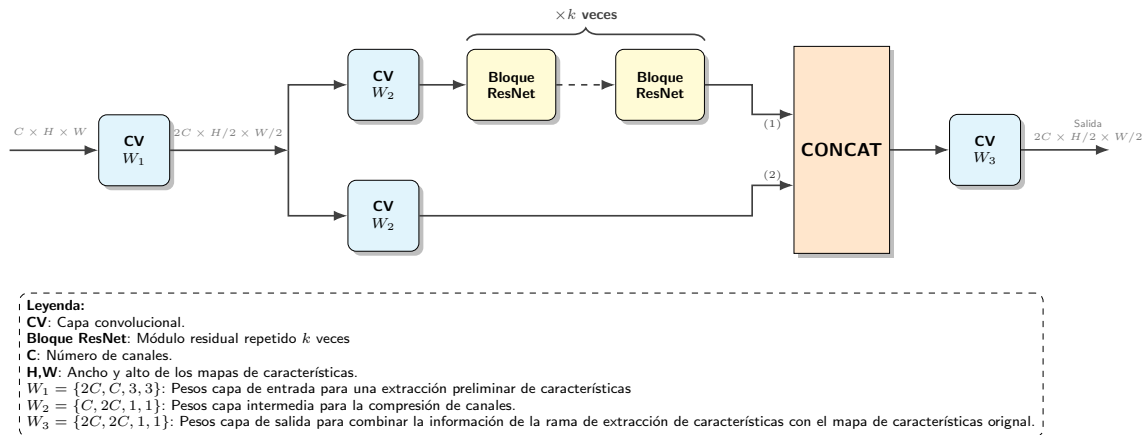


Figura 7: Arquitectura del Módulo CSPResNe(X)t o C3-k.

El siguiente bloque relevante en YOLO es el representado en la figura 8 que es una variación optimizada del módulo SPP descrito en el artículo [17]. Esta variación optimizada se denomina *Spatial Pyramid Pooling Fast* (SPPF). Este módulo, introducido en la arquitectura YOLO [2], tiene como objetivo principal permitir que la red analice simultáneamente el contexto global de la imagen y los detalles locales mediante una estructura en cascada que optimiza la velocidad de procesamiento.

La filosofía original del SPP [17] propone procesar el mapa de características mediante ventanas de pooling

en paralelo con diferentes tamaños de kernel ($k = \{1, 5, 9, 13\}$) para capturar contextos de distinto tamaño. Sin embargo, la variante SPPF logra un resultado matemático equivalente mediante una estructura en cascada más eficiente. En lugar de ejecutar múltiples ramas paralelas, pasa la entrada secuencialmente a través de varias capas de Max Pooling de tamaño 5×5 . El Max Pooling se define como el valor máximo dentro de una ventana deslizante sobre el mapa de características:

$$y_{i,j} = \max_{(m,n) \in \mathcal{R}} x_{i+m,j+n} \quad (6)$$

donde $\mathcal{R}$ es la región del kernel. El Max Pooling aporta invarianza a pequeñas traslaciones y deformaciones. Al quedarse solo con la característica más representativa de una región, el modelo se vuelve robusto ante ligeros desplazamientos del objeto, permitiendo reconocer patrones independientemente de su posición exacta dentro de la ventana.

Para ello, el proceso comienza con la capa convolucional W_1 , que reduce los canales de entrada a la mitad con el fin de disminuir el coste computacional de las operaciones de pooling subsiguientes. Esta etapa da paso a una estructura secuencial en cascada, donde la salida de cada bloque se bifurca iterativamente: una rama alimenta al siguiente bloque y otra se dirige directamente al concatenador. Finalmente, la capa convolucional W_2 integra todas estas proyecciones multiescala, generando una representación unificada y robusta frente a las transformaciones del espacio de características de entrada.

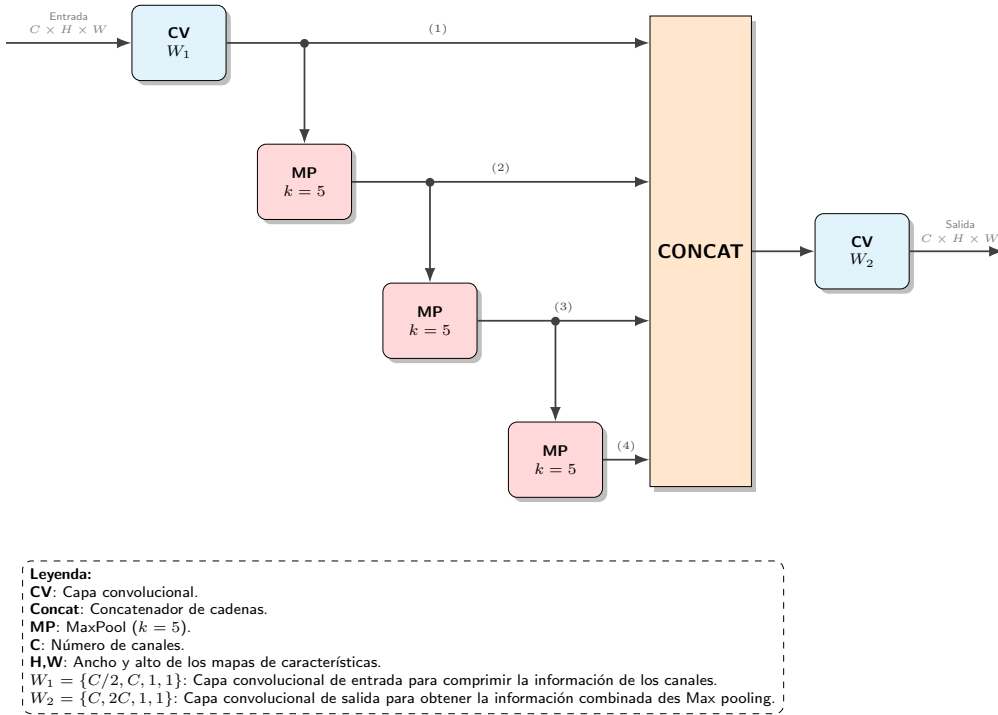


Figura 8: Arquitectura del bloque SPPF.

El último bloque funcional de la red YOLO es la cabeza de detección, representada en la figura 9. Este módulo está constituido por tres ramas independientes que operan en paralelo, cuyas salidas se procesan para generar el vector final con todas las predicciones de la imagen.

La estrategia detrás de este cabezal de procesamiento es el de detección de objetos de diferente tamaño. Para ello, la cabeza de salida procesa mapas de características en tres resoluciones espaciales distintas, denotadas como $i = \{1, 2, 3\}$, donde cada nivel se especializa en un rango de detección específico:

1. **Nivel $i = 1$ (Alta Resolución):** Trabaja con mapas de características de tamaño 80×80 . Al conservar una mayor densidad espacial, esta rama es idónea para la detección de objetos pequeños y detalles finos que se perderían en compresiones mayores.
2. **Nivel $i = 2$ (Resolución Media):** Opera con mapas de 40×40 , sirviendo como un punto de equilibrio para la detección de objetos de tamaño medio.

3. **Nivel $i = 3$ (Baja Resolución):** Utiliza mapas de características reducidos de 20×20 . Aunque la resolución espacial es menor, cada característica en este mapa contiene información condensada de una gran región de la imagen original, siendo la rama responsable de detectar objetos grandes.

Internamente, el diseño del cabezal es el factor determinante que permite clasificar a la arquitectura YOLO como una Red Totalmente Convolutiva (FCN) [18]. Esta configuración marca una diferencia sustancial respecto a los detectores clásicos, los cuales finalizaban con capas totalmente conectadas (estructuras tipo perceptrón) que obligaban a redimensionar cualquier imagen de entrada a un tamaño fijo de salida. Por el contrario, en YOLO se sustituyen estas capas por una convolución final de tamaño 1×1 que elimina esta restricción dimensional.

Esta operación inicial es crítica, ya que actúa como la interfaz de traducción entre el espacio de características latentes y el espacio de predicción. Su función específica es proyectar la profundidad del tensor entrante para que coincida matemáticamente con las dimensiones requeridas en la salida.

A la salida de esta proyección convolutiva, el tensor resultante se reestructura para generar, por cada celda de la rejilla espacial, un total de $B = 3$ predicciones correspondientes a las cajas de anclaje. Cada predicción genera un vector de características que se somete a la función de activación Sigmoide para normalizar los valores en el rango $[0, 1]$. Este vector de salida se descompone en tres segmentos funcionales:

1. **Coordenadas de Caja (4 valores):** Los primeros cuatro elementos (t_x, t_y, t_w, t_h) representan las coordenadas relativas al anchor y a la celda de la rejilla. Estas se transforman mediante una función de decodificación para obtener las coordenadas absolutas (x, y) del centro y las dimensiones (w, h) de la caja predicha.
2. **Probabilidad de Objeto (1 valor):** Representa la confianza de que exista un objeto dentro de la caja propuesta.
3. **Probabilidades de Clase (C valores):** Las columnas restantes contienen la probabilidad condicional de que el objeto detectado pertenezca a una etiqueta específica.

Finalmente, el proceso concluye con la concatenación y el reescalado de las predicciones. Las dimensiones del tensor se ajustan permitiendo que todos los vectores de salida, independientemente de la escala de la que provengan, se unifiquen en una única lista predicciones.

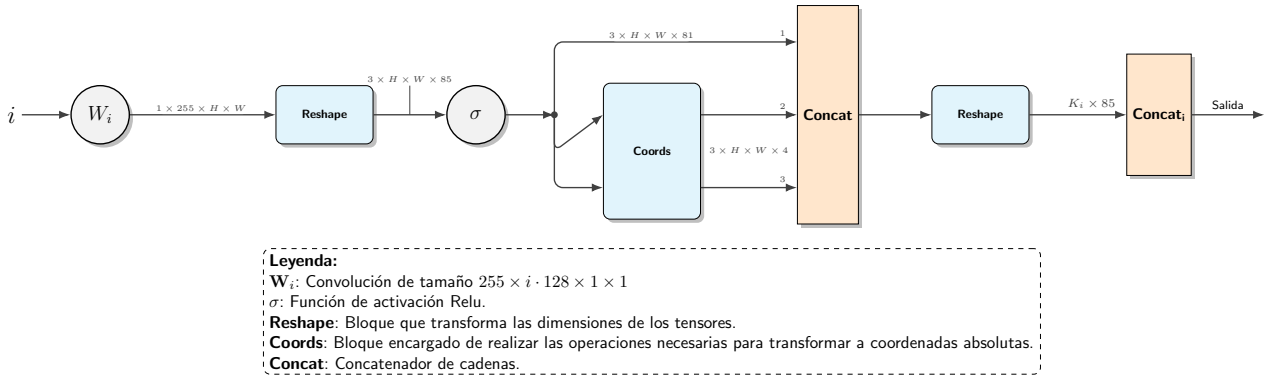


Figura 9: Detalle de la Cabeza de Detección.

Por último, se detalla la arquitectura global representada en la Figura 10. Este diseño constituye una evolución directa de las metodologías propuestas en la familia de detectores YOLO a través de los artículos descritos anteriormente. Aunque este esquema específico corresponde a la implementación de YOLOv5, su diseño modular se fundamenta en ir integrando los avances definidos en la literatura científica previamente, comenzando con la versión original del detector de una sola etapa [8] hasta las optimizaciones de flujo de gradiente de la versión 4 [14].

La red se estructura secuencialmente en tres etapas funcionales descritas en detalle en la literatura: Backbone (esqueleto), Neck (cuello) y Head (cabeza), cuyo análisis detallado se presenta a continuación basándose en el diagrama de la figura 10.

2.2.1. Backbone: Extracción de Características

El proceso comienza con una entrada estándar de $3 \times 640 \times 640$. No obstante, debido a que la red es FCN, es posible procesar imágenes de dimensiones variables ya que no existen capas densas que fijen el tamaño de entrada. Es relevante destacar también que aunque la arquitectura base está diseñada para imágenes RGB (3 canales), para la aplicación en imágenes térmicas se modifica la primera capa convolucional. Al ajustar la capa de entrada de tres canales a uno, se adapta la red a la naturaleza monocanal de estas imágenes para así reducir la redundancia en las operaciones.

La primera etapa de procesamiento es realizada por la capa convolucional W_1 , que reduce la resolución espacial inicial para disminuir el coste computacional. A diferencia de las topologías secuenciales clásicas como Darknet-19 o Darknet-53 [12] [13], que se basan en el apilamiento lineal de capas convolucionales y max-pooling siguiendo el paradigma convencional visto en clase, el esqueleto propuesto integra bloques C3-k.

Esta estructura, introducida en el artículo de YOLOv4 [14], divide el flujo de características en dos ramas para evitar el cálculo redundante de gradientes, así como estabilizarlos mediante las ramas residuales. Como se observa en el diagrama, los bloques C3-1, C3-2 y C3-3 actúan como extractores profundos, permitiendo que la red aprenda características complejas sin saturar el flujo de retropropagación. El backbone finaliza con el módulo SPPF diseñado para separar las características de contexto más importantes antes de ingresar al cuello de la red.

2.2.2. Neck: Fusión de Escalas

La sección intermedia o Neck es la responsable de la detección multiescala. Tal y como se introdujo en YOLOv3 [13], es fundamental realizar predicciones en diferentes escalas para detectar eficazmente tanto objetos pequeños como grandes. Sin embargo, la arquitectura de la Figura 10 mejora la propuesta original de la versión 3 implementando una estructura PANet (Path Aggregation Network) mediante los bloques CSP, característica clave adoptada desde YOLOv4 [14].

El diagrama muestra explícitamente este flujo bidireccional:

1. **Ruta Descendente:** Los mapas de características profundos (con muchas características pero con baja resolución espacial) provenientes de W_2 se procesan y redimensionan (bloque Resize) para concatenarse con las capas previas del backbone. Al concatenarse con las características previas del backbone, se dota a los mapas de alta resolución (encargados de detectar objetos pequeños) de la capacidad de contexto necesaria para clasificar correctamente.
2. **Ruta Ascendente:** Posteriormente, y de forma simétrica, se aprovecha que las capas de mayor resolución conservan muy bien los detalles de bordes y contornos. Esta información se envía hacia las capas de menor resolución a través de las convoluciones de reducción W_4 y W_5 . El objetivo es afinar la precisión de las cajas delimitadoras, mejorando la localización incluso en las ramas que detectan objetos grandes.

Esta estrategia de agregación de rutas garantiza que las tres salidas del cuello contengan una mezcla óptima de información contextual y espacial.

2.2.3. Head: Detección

Finalmente, las tres ramas resultantes alimentan a la Cabeza de Detección. Siguiendo el paradigma establecido en YOLOv2 [12], la red no predice las cajas delimitadoras directamente desde cero, sino que ajusta desplazamientos sobre cajas de anclaje predefinidas.

Como se observa en la parte inferior de la Figura 10, el sistema genera tres salidas independientes, una estrategia consolidada en YOLOv3 [13] para manejar la variación de tamaño de los objetos:

- La rama de la izquierda (alta resolución) se especializa en objetos pequeños.
- La rama central (resolución media) detecta objetos medianos.

- La rama de la derecha (baja resolución) se encarga de los objetos grandes.

El tensor de salida final unificado ($1 \times 25200 \times 85$) es el resultado de aplanar y concatenar estas predicciones multiescala, permitiendo que el modelo mantenga la velocidad de inferencia en tiempo real característica de la familia YOLO [8] mientras maximiza la precisión en diversas escalas.

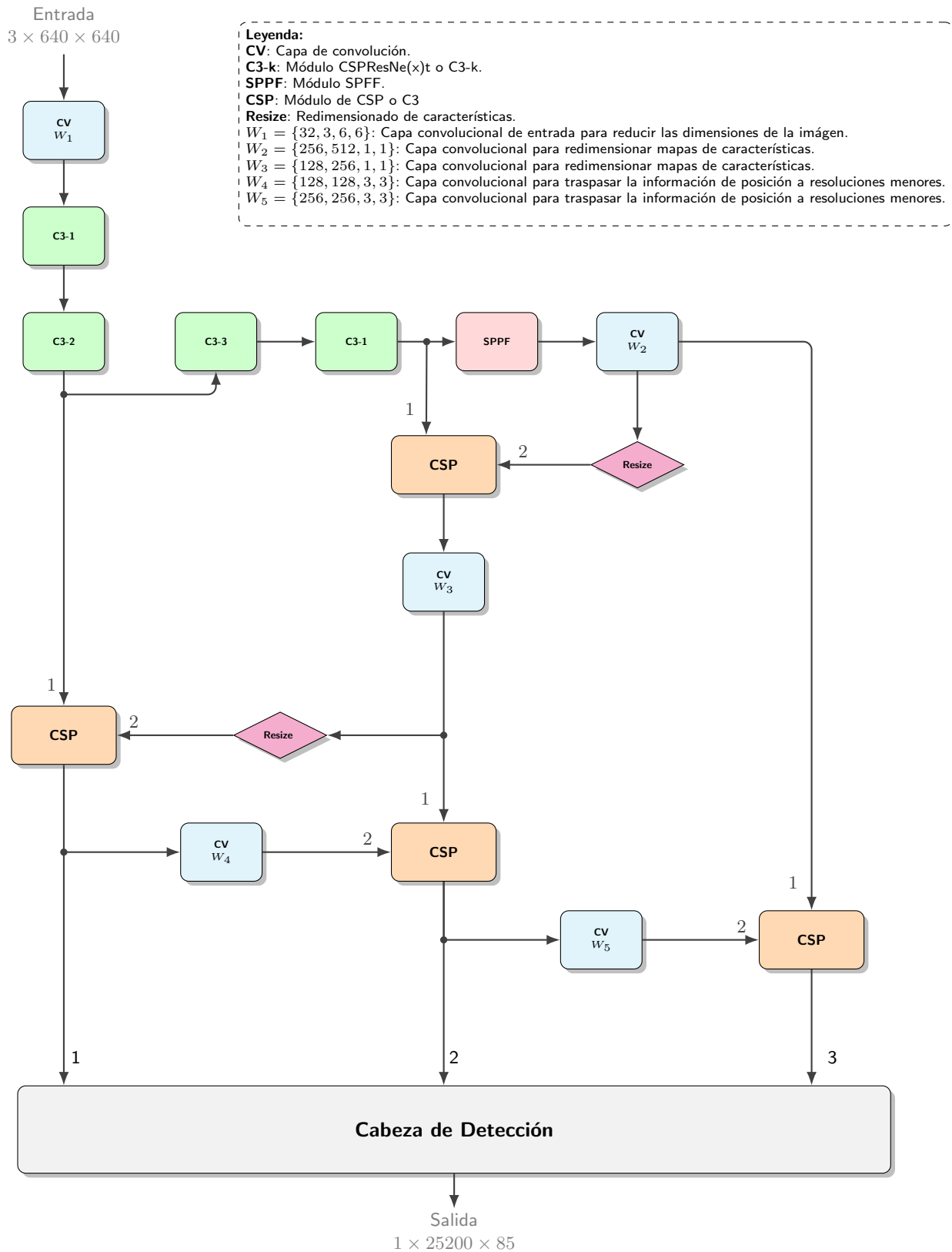


Figura 10: Arquitectura Global YOLO: Inicio Vertical y Flujo Descendente

2.3. Reglas de entrenamiento

El entrenamiento se realiza mediante el cálculo de la Intersección sobre la Unión (IoU) entre la predicción y el *ground truth* térmico. Se utilizan funciones de pérdida como CIoU que consideran la distancia entre centros, la escala y el aspecto de la caja de detección.

2.3.1. Entrenamiento de la red

La red YOLO se entrena mediante la técnica de retropropagación. Primero se realiza una pasada hacia delante en la que se calcula la salida de la red. Luego, se actualizan los pesos y sesgos por descenso del gradiente estocástico empezando por las capas finales y acabando en las iniciales. La función de pérdida a minimizar es de la forma [8]:

$$\begin{aligned}
\mathcal{L} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned} \tag{7}$$

Donde $\mathbb{1}_{i,j}^{\text{obj}}$ indica si un objeto aparece en la j -ésima caja de la i -ésima celda, x_i e y_i son las coordenadas del centro del objeto, w_i y h_i el ancho y el alto del objeto. La red tiene una confianza $\hat{C}_i = P(\text{Objeto}) \cdot \text{IoU}$ de que la caja que ha estimado contiene un objeto, que depende de la probabilidad que asigna a la existencia de dicho objeto. El valor real de la confianza será 0 si no hay objeto o la intersección sobre la unión si lo hay (si fuese 1 se estarían penalizando doblemente los errores de detección). Por último, $\hat{p}_i(c)$ corresponde a un término de clasificación, que representa la probabilidad que la red asigna a que el objeto pertenezca a una determinada clase.

Los errores de detección en objetos grandes son menos graves que en los pequeños, por lo que se toma la raíz cuadrada en los términos de detección del ancho y alto para tener en cuenta este efecto. Además, se observan 2 hiperparámetros: λ_{coord} y λ_{noobj} . Ambos tienen el propósito de incentivar a la red a predecir cajas aunque se equivoque. Dado que la mayoría de las celdas no contienen objetos, el término correspondiente en la función de pérdida es dominante y YOLO tiende a decir que no hay ningún objeto. Con λ_{coord} se aumenta la penalización al cometer errores de localización y con λ_{noobj} se reduce la importancia de predecir cajas cuando no las hay.

La formulación anterior corresponde a YOLOv1. Aunque en este trabajo se esté usando YOLOv5, se considera que dicha ecuación proporciona una buena intuición sobre el funcionamiento de la red. En versiones posteriores se han introducido algunas modificaciones y refinamientos sobre esta idea. La función de pérdida en YOLOv5 considera la entropía cruzada binaria para las pérdidas de clasificación y existencia de objetos, y la intersección sobre la unión completa (CIoU) para las de localización:

$$\begin{aligned}
\mathcal{L} = & \lambda_{\text{loc}} \mathcal{L}_{\text{loc}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} \\
= & \lambda_{\text{loc}} (1 - \text{CIoU}) + \lambda_{\text{obj}} \text{BCE}_{\text{obj}} + \lambda_{\text{cls}} \text{BCE}_{\text{cls}}
\end{aligned} \tag{8}$$

Donde λ_{loc} , λ_{obj} y λ_{cls} se corresponden con los hiperparámetros **box**, **obj** y **cls**, respectivamente.

En vez de estimar directamente las coordenadas del centro del objeto, el ancho y el alto, lo hace de forma relativa a la posición de cada celda[2]:

$$b_x = [2\sigma(t_x) - 0,5] + c_x \quad (9)$$

$$b_y = [2\sigma(t_y) - 0,5] + c_y \quad (10)$$

$$b_w = p_w [2\sigma(t_w)]^2 \quad (11)$$

$$b_h = p_h [2\sigma(t_h)]^2 \quad (12)$$

Donde c_x y c_y son las coordenadas de la esquina superior izquierda de las celdas. Las salidas de las posiciones de los centros de los objetos se pasan por una sigmoide, de forma que las variaciones $2\sigma(t) - 0,5$ toman valores entre -0.5 y 1.5 (siendo 0 la esquina superior izquierda y 1 la esquina superior derecha). La razón de no restringirlas entre 0 y 1 tiene que ver con que estos valores se alcanzan para un t muy grande, dificultando la detección en los extremos de las celdas. A diferencia de otras versiones donde el centro de los objetos quedaba restringido a los límites de cada celda, YOLOv5 permite que se desplace ligeramente fuera de dichos límites. Asimismo, se sigue resolviendo el problema de inestabilidad en el entrenamiento debido a la falta de restricciones espaciales que provocaba que las celdas pudieran detectar objetos en cualquier parte de la imagen[12].

Los parámetros p_w y p_h corresponden a unos anchos y altos predeterminados que facilitan que la red no tenga que aprender las dimensiones de los objetos desde cero. Estos se seleccionan mediante el algoritmo k-means sobre las cajas del conjunto de entrenamiento para representar las formas más probables de los objetos. El número de centroides se selecciona donde se sitúe el codo en la curva IoU frente a número de *clusters*[12] y se indica en el hiperparámetro **anchors**. El hiperparámetro **anchor_t** establece el factor de escala máximo permitido entre las dimensiones de las cajas predeterminadas y las cajas reales. Si la diferencia de tamaño excede este multiplicador en un porcentaje determinado de objetos, el conjunto de datos no queda bien representado por las cajas predeterminadas, lo que activa su redimensionado automático para ajustarse mejor a la realidad del *dataset*.

El multiplicador $[2\sigma(t)]^2$ reescala las cajas predefinidas para ajustarse al objeto detectado. A diferencia de versiones anteriores en las que se usaba la exponencial (no acotada), los valores que adopta se restringen entre 0 y 4 para que no exploten las predicciones.

Por último, cabe notar que las salidas b están en valores relativos al tamaño de la imagen, por lo que deben reescalarsse para obtener los valores en píxeles de x , y , w y h .

2.3.2. Hiperparámetros

Los hiperparámetros se organizan según se relacionen con la optimización del entrenamiento, la fase de calentamiento, la función de pérdida, el proceso de *data augmentation* o la evaluación de la red.

Optimización del entrenamiento

La actualización de pesos se basa en el método del gradiente:

$$\theta_{i+1} = \theta_i - \varepsilon \nabla_{\theta} \mathcal{L}(\theta_i) \quad (13)$$

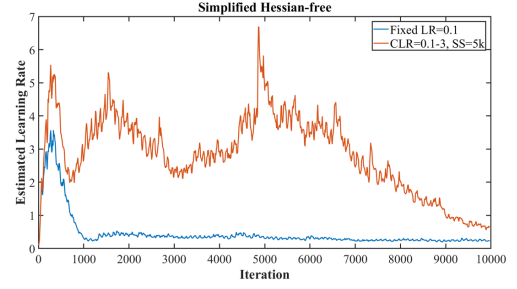
Donde θ_i son los pesos de la iteración i -ésima, ε es la tasa de aprendizaje y $\mathcal{L}(\theta)$ representa la función de pérdida. Esta última puede aproximarse localmente por un desarrollo de Taylor de segundo orden:

$$\mathcal{L}(\theta_i - \varepsilon \nabla_{\theta} \mathcal{L}(\theta_i)) \approx \mathcal{L}(\theta_i) + (\theta_{i+1} - \theta_i)^T \nabla_{\theta} \mathcal{L}(\theta_i) + \frac{1}{2} (\theta_{i+1} - \theta_i)^T H(\theta_{i+1} - \theta_i) \quad (14)$$

El cálculo directo de la matriz Hessiana, H , es computacionalmente costoso debido al elevado número de parámetros en las redes. No obstante, bajo el concepto de *Hessian-free optimization*, no se requiere la matriz completa, sino solamente una aproximación por diferencias finitas en la dirección del gradiente. La tasa de aprendizaje óptima para cada peso puede obtenerse a partir de la tasa real utilizada:



(a) Exactitud de una Resnet-56 en función de la tasa de aprendizaje con 100000 (amarillo) iteraciones, 20000 iteraciones (rojo) y 5000 iteraciones (azul). Se observa que la exactitud es significativamente inferior con pocas iteraciones para tasas de aprendizaje pequeñas. Sin embargo, para tasas de aprendizaje grandes, todas las curvas convergen a un valor de exactitud similar independientemente del número de iteraciones.



(b) Tasa de aprendizaje óptima según el número de iteraciones al usar una tasa de aprendizaje fija (azul) y CLR (rojo). En el primer caso, la tasa óptima comienza en valores elevados, pero rápidamente decrece y se estabiliza en valores bajos. Esto se puede interpretar como que el optimizador ha quedado atrapado en zonas de alta curvatura donde pasos grandes del gradiente provocarían inestabilidad. En el segundo caso, la tasa óptima se mantiene elevada durante la mayor parte del proceso. Inicialmente, las tasas de aprendizaje altas permiten sortear mínimos locales ruidosos (hessiano elevado) y converger hacia valles planos y anchos (hessiano bajo) en los que puede avanzar a grandes pasos sin estancarse.

Figura 11: Gráficas de superconvergencia extraídas de [19].

$$\varepsilon^* = \varepsilon \frac{\theta_{i+1} - \theta_i}{2\theta_{i+1} - \theta_i - \theta_{i+2}} \quad (15)$$

Se puede obtener una estimación de la tasa de aprendizaje óptima global (es decir, para la red como conjunto en lugar de para cada peso individual) calculando el cociente entre las sumas de los valores absolutos de los numeradores y denominadores de la expresión anterior para todos los pesos.

La tasa de aprendizaje es un hiperparámetro fundamental en el entrenamiento de las redes. Algunas estrategias la establecen a un valor constante durante todo el entrenamiento o se reduce progresivamente a medida que avanza el entrenamiento. Otras proponen variarla entre un valor mínimo y uno máximo para explorar de forma más eficiente la función de pérdida. Esta se conoce como tasa de aprendizaje cíclica (CLR, por sus siglas en inglés). El proceso consta de dos etapas que se repite alternamente: una primera en la que se incrementa la tasa de aprendizaje, y una segunda en la que se disminuye. El número de iteraciones de cada etapa viene dado por el tamaño de paso.

En [19] proponen una modificación sobre CLR, en la que solamente se realiza un ciclo de subida-bajada de una duración menor que el total de épocas y reduciendo la tasa de aprendizaje hasta varios órdenes de magnitud la tasa inicial en las épocas restantes. Este comportamiento busca lograr la super-convergencia, la cual hace referencia al uso de tasas de aprendizaje inusualmente elevadas que permiten acelerar el entrenamiento hasta varios órdenes de magnitud respecto de los métodos estándar. La figura 11 ilustra este fenómeno.

A nivel de implementación, el CLR se estructura en dos etapas:

- Calentamiento (*warmup*): es la fase inicial del CLR, en la que la tasa de aprendizaje empieza en niveles bajos y sube gradualmente hasta alcanzar la tasa inicial. Evita el sobreajuste temprano hacia los primeros lotes de datos procesados, al restringir el tamaño de los pasos del gradiente. El hiperparámetro `warmup_epochs` indica el número de épocas de calentamiento.

Durante esta fase, la actualización de la tasa de aprendizaje sigue la dinámica:

$$\varepsilon = \varepsilon_0 + \frac{n_i}{n_w}(\varepsilon_1 lr_0 - \varepsilon_0) \quad (16)$$

$$\varepsilon_1 = \left(1 - \frac{ep}{Ep}\right)(1 - lr_f) + lr_f \quad (17)$$

$$n_i = i + ep \cdot n_b \quad (18)$$

$$n_w = \text{máx}(\text{warmup_epochs} \cdot n_b, 100) \quad (19)$$

Donde ε_0 es la tasa inicial de aprendizaje (se puede modificar en los sesgos con el hiperparámetro `warmup_bias_lr`), Ep el total de épocas, ep el número de la época actual (empezando en 0), n_w el número de iteraciones de calentamiento, n_i el número de iteración actual, n_b el número de lotes en una época.

Además, aparecen los hiperparámetros de control lr_0 y lr_f . El primero proporciona una referencia de la tasa inicial (de la fase de entrenamiento) objetivo a la que se quiere llegar al final del calentamiento. Cabe notar que esta tasa no es el objetivo como tal, sino que viene modulada por el número de la época. El número de iteraciones de calentamiento es en general menor que el número de iteraciones en una época. En tal caso, $\varepsilon_1 = 1$ y la tasa de aprendizaje al final del calentamiento es lr_0 . En caso contrario (si se fija un valor diferente de 1 en `warmup_epochs`), la tasa de aprendizaje acaba el calentamiento en un valor entre lr_0 y $lr_0 \cdot lr_f$. El segundo se refiere a un factor de escala de la tasa de aprendizaje, de forma que esta acaba en $lr_0 \cdot lr_f$ al final de la fase de entrenamiento.

De las ecuaciones anteriores se deduce que el crecimiento de la tasa de aprendizaje durante el calentamiento sigue una curva que se ralentiza según avanza las épocas. Se trata de una interpolación lineal entre ε_0 y $\varepsilon_1 \cdot lr_0$, donde el término ε_1 va disminuyendo con las épocas, por lo que cada vez los incrementos (n_i sigue creciendo) en la tasa de aprendizaje son menores.

- Entrenamiento: corresponde con la segunda fase del CLR, en la que la tasa de aprendizaje decae desde el valor alcanzado en el calentamiento hasta el valor $lr_0 lr_f$ según la dinámica:

$$\varepsilon = \varepsilon_0 \left[\left(1 - \frac{ep}{Ep}\right)(1 - lr_f) + lr_f \right] \quad (20)$$

- Fuerza de la inercia en la actualización de gradientes (`momentum`):
- Penalización para la regularización de los pesos (`weight_decay`):

Fase de calentamiento (*Warmup*) Configuraciones temporales para estabilizar el modelo en las iteraciones iniciales.

- `warmup_epochs`: Duración de la fase de estabilidad inicial.
- `warmup_momentum`: Momentum reducido para el arranque.
- `warmup_bias_lr`: Tasa de aprendizaje específica para los sesgos iniciales.

Función de pérdida Determinan la importancia relativa de cada error cometido por el modelo.

- `box`: Peso del error en la localización de las cajas.
- `obj`: Importancia del error en la detección de presencia de objetos.
- `cls`: Peso del error en la clasificación de las categorías.

Data augmentation Parámetros que rigen las transformaciones aleatorias para mejorar la robustez.

- *Color*: Ajustes de tono (`hsv_h`), saturación (`hsv_s`) y brillo (`hsv_v`).
- *Geometría*: Rotación (`degrees`), traslación (`translate`) y escalado (`scale`).
- *Mezcla*: Probabilidades de técnicas como `mosaic` o `mixup`.

Evaluación de la red Parámetros que rigen las transformaciones aleatorias para mejorar la robustez.

- *Color*: Ajustes de tono (**hsv_h**), saturación (**hsv_s**) y brillo (**hsv_v**).
- *Geometría*: Rotación (**degrees**), traslación (**translate**) y escalado (**scale**).
- *Mezcla*: Probabilidades de técnicas como **mosaic** o **mixup**.

3. Desarrollo: Aplicación en Imágenes Térmicas

Para usar YOLO con imágenes térmicas, el proceso técnico seguido es:

1. **Conversión de Datos**: Transformación de las lecturas de temperatura radiométrica a una escala de grises de 8 bits para compatibilidad con la arquitectura estándar.
2. **Aumento de Datos Especializado**: Inserción de ruido gaussiano para simular condiciones atmosféricas adversas que afectan la señal infrarroja.
3. **Entrenamiento**: Ajuste del modelo sobre un dataset de 640×640 píxeles, optimizando los pesos para detectar firmas térmicas en entornos nocturnos.

4. Resultados

Los resultados muestran que el modelo YOLO es capaz de distinguir entre un ser vivo y un objeto inanimado con una temperatura similar gracias al análisis morfológico.

En el caso general:

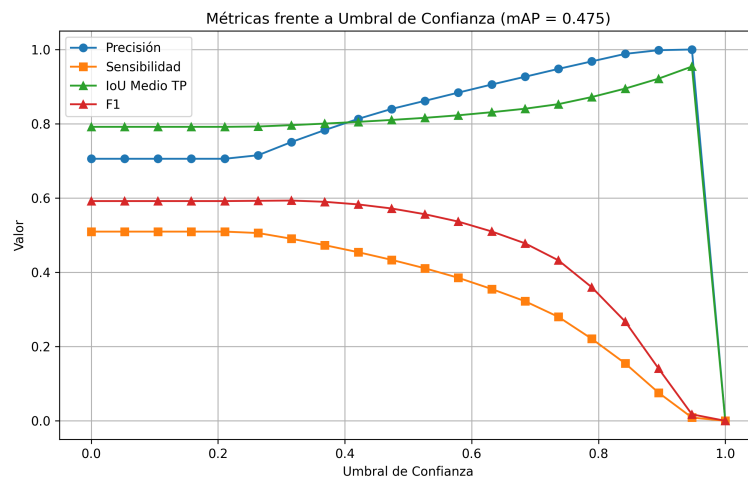
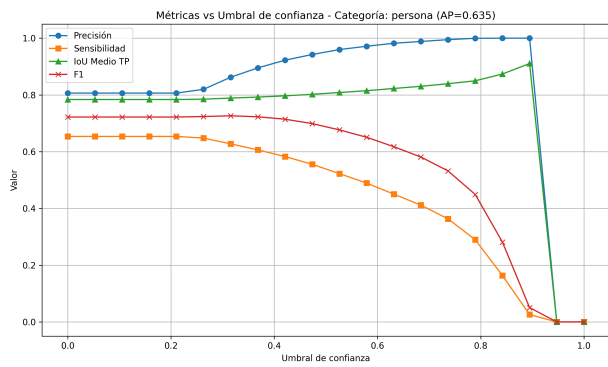
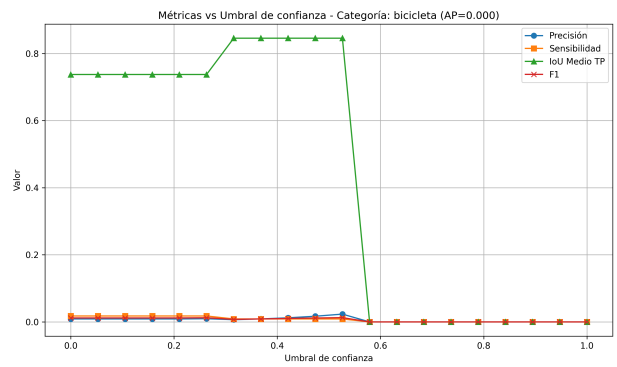


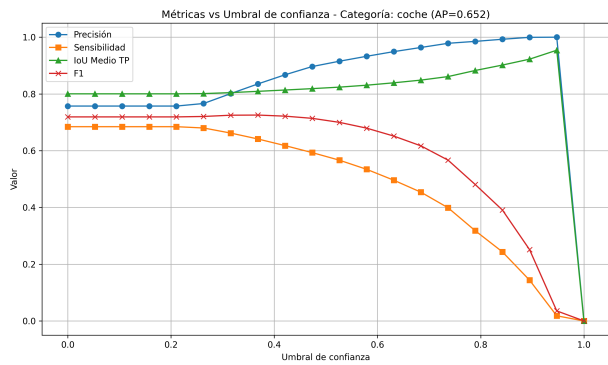
Figura 12: Métricas globales del clasificador.



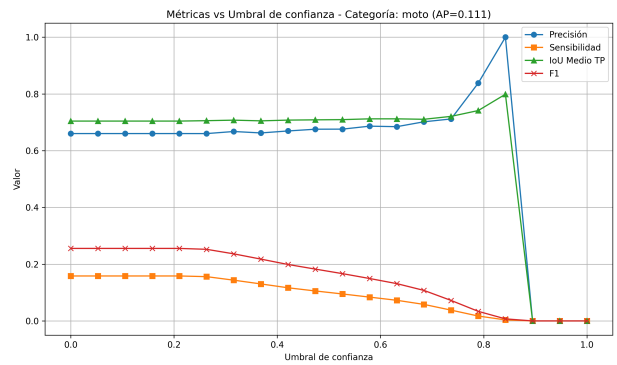
(a) Métricas para la clase persona.



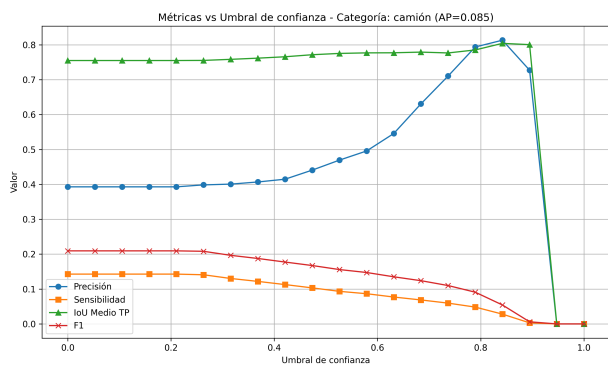
(b) Métricas para la clase bicicleta.



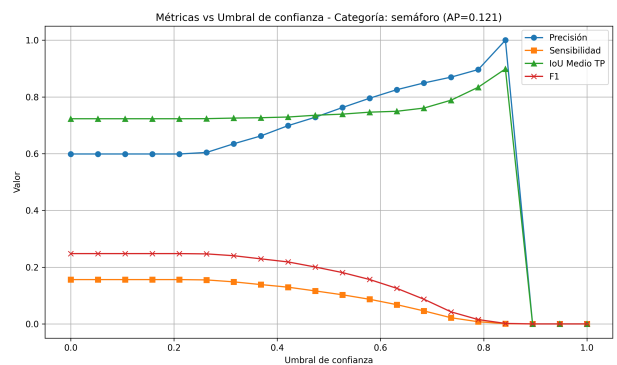
(c) Métricas para la clase coche.



(d) Métricas para la clase moto.

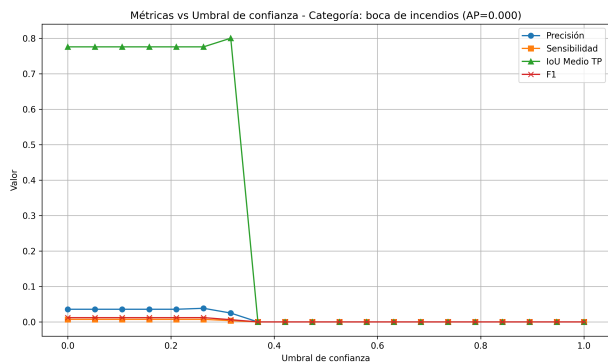


(e) Métricas para la clase camión.

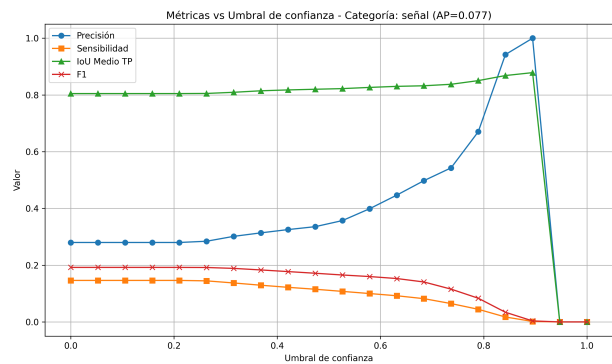


(f) Métricas para la clase semáforo.

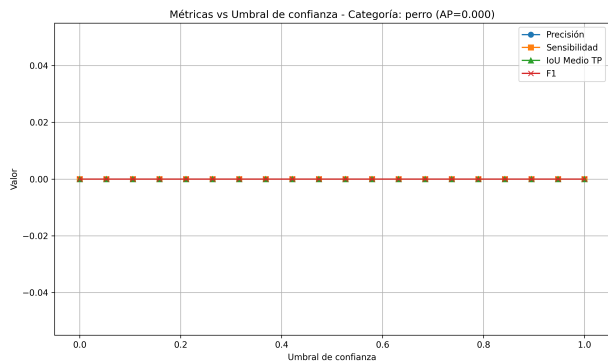
Figura 13: Métricas de rendimiento del clasificador para las categorías: persona, bicicleta, coche, moto, camión y semáforo.



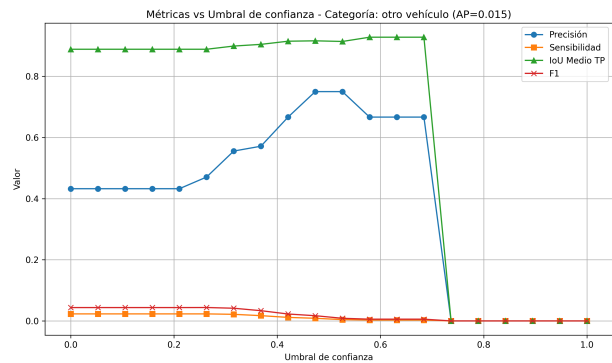
(a) Métricas para la clase boca de incendios.



(b) Métricas para la clase señal.



(c) Métricas para la clase perro.



(d) Métricas para la clase otros vehículos.

Figura 14: Métricas de rendimiento del clasificador para las categorías: boca de incendios, señal, perro y otros vehículos.

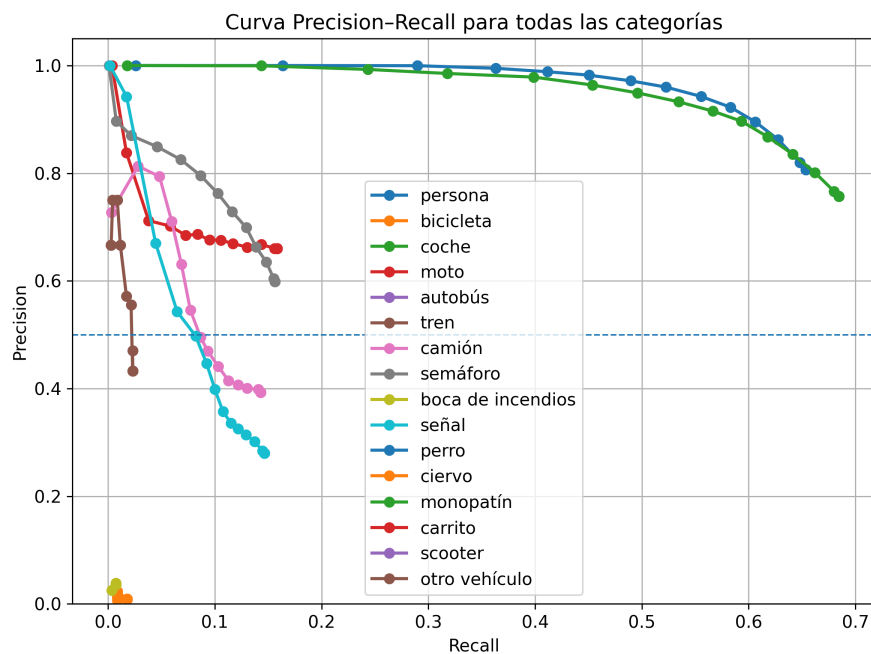


Figura 15: Curva precisión recall.

Referencias

- [1] C. Jiang, H. Ren, X. Ye, J. Zhu, H. Zeng, Y. Nan, M. Sun, X. Ren, and H. Huo, "Object detection from uav thermal infrared images and videos using yolo models," *International Journal of Applied Earth Observation*

and *Geoinformation*, vol. 112, p. 102912, 2022.

- [2] G. Jocher, “YOLOv5 by Ultralytics,” 5 2020.
- [3] Teledyne FLIR, “Free adas thermal dataset v2.” <https://www.flir.com/oem/adas/adas-dataset-form/>, 2022. Accessed: 2024-02-08.
- [4] Y. Munian, A. Martinez-Molina, D. Miserlis, H. Hernandez, and M. Alamaniotis, “Intelligent system utilizing hog and cnn for thermal image-based detection of wild animals in nocturnal periods for vehicle safety,” *Applied Artificial Intelligence*, vol. 36, no. 1, p. 2031825, 2022.
- [5] Y. Qian, C. Zhen, M. Wang, S. Han, J. Li, C. Zhao, and T. Zhu, “An improved hog-svm recognition algorithm of power equipment with infrared images,” *Journal of Physics: Conference Series*, vol. 2010, p. 012119, sep 2021.
- [6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” 2016.
- [7] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, *SSD: Single Shot MultiBox Detector*, p. 21–37. Springer International Publishing, 2016.
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016.
- [9] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” 2018.
- [10] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” 2017.
- [11] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” 2017.
- [12] J. Redmon and A. Farhadi, “Yolo9000: Better, faster, stronger,” 2016.
- [13] J. Redmon and A. Farhadi, “Yolov3: An incremental improvement,” 2018.
- [14] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” 2020.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015.
- [16] C.-Y. Wang, H.-Y. M. Liao, I.-H. Yeh, Y.-H. Wu, P.-Y. Chen, and J.-W. Hsieh, “Cspnet: A new backbone that can enhance learning capability of cnn,” 2019.
- [17] K. He, X. Zhang, S. Ren, and J. Sun, *Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition*, p. 346–361. Springer International Publishing, 2014.
- [18] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” 2015.
- [19] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates. arxiv,” *arXiv preprint arXiv:1708.07120*, vol. 6, 2017.